Project Baseline

Project Baseline

Metrics

Metric	Value	Rating
Performance	45	🔴 Poor
Accessibility	91	🟢 Good
Best Practices	77	🟡 Needs Improvement
SEO	100	🟢 Good
First Contentful Paint (FCP)	0.7 s	🟢 Good
Largest Contentful Paint (LCP)	2.6 s	🟡 Needs Improvement
Speed Index	12.9 s	🔴 Poor
Total Blocking Time (TBT)	6,660 ms	🔴 Poor

Cumulative Layout Shift (CLS)	0.004	🟢 Good

The Lighthouse audit shows that rendering performance is the weakest area of the website. While Accessibility and SEO perform well, the low Performance score indicates significant opportunities to improve loading speed and responsiveness.

Networking

Initial (Hard) Reload

Metric	Value
Requests	**771**
Data Transferred	**8.2 MB**
Total Resource Size	**72.2 MB**
Load Time	**6.3 s**

Soft Reload

Metric	Value
Requests	**628**
Data Transferred	**635 kB**
Total Resource Size	**92.3 MB**
Load Time	**5.4 s**

Cache Savings

- Initial transfer: **8.2 MB**
- Repeat transfer: **635 kB**

- **≈92% reduction in transferred data**

This demonstrates that browser caching is effective for returning visitors.

Resource Breakdown

Resource	Size	Approx. Share
JavaScript	~25.0 MB	~35%
CSS	~2.2 MB	~3%
Images	~1.0 MB	~1.5%

JavaScript represents by far the largest resource category and contributes significantly to download, parsing, and execution time. JavaScript assets are compressed with gzip, reducing network transfer costs.

Corrective Findings

Finding 1 — Excessive JavaScript Payload and Main-Thread Work

Observation

The homepage loads approximately **25 MB of JavaScript**, making it the largest resource category. Lighthouse also reports a **Total Blocking Time of 6,660 ms**, indicating extensive JavaScript execution before the page becomes fully interactive.

User Impact

Users may see page content but experience delays when scrolling, clicking buttons, opening menus, or interacting with the website.

Metrics Affected

- Total Blocking Time (TBT)
- Performance Score
- Largest Contentful Paint (LCP)
- Interaction responsiveness

Most Likely Cause

Large JavaScript bundles and significant client-side processing during the initial page load.

Recommended Solution

- Remove unused JavaScript.
- Split large bundles into smaller chunks.
- Defer non-essential scripts.
- Lazy-load functionality that is not required immediately.

Finding 2 — Images Are Larger Than Necessary

Observation

The audit estimates **1,596 KiB** of potential image savings. Several images are downloaded at much higher resolutions than required.

User Impact

Large images increase loading time, particularly on slower networks and mobile devices.

Metrics Affected

- Largest Contentful Paint (LCP)
- Speed Index
- Performance Score

Most Likely Cause

Oversized images and insufficient image compression.

Recommended Solution

- Compress images further.
- Use WebP or AVIF.
- Serve appropriately sized images for each device.

Finding 3 — Excessive Number of Network Requests

Observation

The homepage generates **771 requests** during the initial load.

User Impact

Each request introduces additional network overhead and latency, particularly for mobile users.

Metrics Affected

- Overall page load time
- Speed Index
- LCP

Most Likely Cause

Many JavaScript files, third-party services, API calls, fonts, and other assets.

Recommended Solution

- Remove unnecessary assets.
- Combine small resources where appropriate.
- Delay non-critical requests until after the initial page load.

Finding 4 — Initial Download Size Is High

Observation

The initial page load transfers **8.2 MB** of data.

User Impact

Large downloads increase loading time and consume more mobile data.

Metrics Affected

- Largest Contentful Paint (LCP)

- Speed Index
- Performance Score

Most Likely Cause

Large JavaScript bundles, media assets, and third-party resources.

Recommended Solution

Reduce JavaScript size, optimize media assets, and remove unnecessary resources.

Finding 5 — Browser Cache Lifetime Can Be Improved

Observation

Although caching works well, Lighthouse estimates an additional **1,128 KiB** could be saved by improving cache lifetimes.

User Impact

Returning users still re-download assets that could remain cached longer.

Metrics Affected

- Largest Contentful Paint (LCP)
- Speed Index

Most Likely Cause

Short cache lifetimes for some static assets.

Recommended Solution

Increase cache durations for static images, CSS, JavaScript, and fonts where appropriate.

Finding 6 – Overall Rendering Performance Remains Poor

Observation

The Lighthouse audit reports an overall **Performance score of 45**, which falls within the **Poor** category. While the website performs well in Accessibility (91) and SEO (100), rendering performance remains significantly below recommended levels, indicating that users may experience slow loading and reduced responsiveness.

User Impact

Users may experience slower page loads and delayed responsiveness, particularly on slower internet connections or less powerful devices. This can negatively affect the overall browsing experience and increase the likelihood of users leaving the site before it becomes fully usable.

Metrics Affected

- Performance Score
- Largest Contentful Paint (LCP)
- Speed Index
- Total Blocking Time (TBT)

Most Likely Cause

The low performance score is primarily the result of several combined issues, including heavy JavaScript execution, oversized images, a high number of network requests, and a large initial download size.

Recommended Solution

Prioritize the improvements identified in this report by reducing JavaScript execution, optimizing image delivery, minimizing unnecessary network requests, and reducing the overall amount of data transferred during the initial page load.

Good Findings

Good Finding 1 — Effective Browser Caching

A soft refresh transfers only **635 kB** compared with **8.2 MB** during the initial load, representing approximately **92% less transferred data**. This demonstrates that browser caching is functioning effectively and provides much faster repeat visits.

Good Finding 2 — HTTP Compression Is Enabled

Response headers indicate that JavaScript assets are compressed using **gzip**, significantly reducing the amount of data transferred over the network. This improves loading performance, especially for users on slower connections.

Overall Conclusion

The Wayfair homepage demonstrates strong Accessibility and SEO scores, along with effective browser caching and HTTP compression. However, overall performance is limited by heavy JavaScript execution, large initial network transfers, oversized images, and a very high number of network requests. Addressing these issues would improve loading speed, responsiveness, and the overall user experience, particularly for mobile users and those on slower network connections.

Mobile Metrics

Mobile Metrics

Metric	Value	Status
Performance	**31**	🔴 Poor
Accessibility	**90**	🟢 Good
Best Practices	**77**	🟡 Needs Improvement
SEO	**100**	🟢 Excellent
First Contentful Paint	**2.9 s**	🟡 Needs Improvement
Largest Contentful Paint	**12.8 s**	🔴 Poor
Total Blocking Time	**9,240 ms**	🔴 Poor
Speed Index	**28.8 s**	🔴 Poor
Cumulative Layout Shift	**0**	🟢 Good

Finding 1 — JavaScript execution is extremely heavy on mobile

The report shows approximately **19 seconds** of JavaScript execution time, contributing to a **Total Blocking Time of 9,240 ms**.

User Impact

Mobile users experience delayed interactions because slower mobile CPUs take much longer to execute large JavaScript bundles.

Metrics affected

- Total Blocking Time (TBT)
- Largest Contentful Paint (LCP)

- Performance Score

Most likely cause

Large JavaScript bundles and extensive client-side processing.

Likely solution

Reduce unused JavaScript, split bundles, defer non-critical scripts, and lazy-load functionality.

Finding 2 — Mobile rendering is delayed by render-blocking resources

The audit estimates that removing render-blocking resources could save about **1.8 seconds**, and reports a **Largest Contentful Paint of 12.8 seconds**.

User Impact

Mobile users wait much longer before the main content becomes visible, especially on Slow 4G connections.

Metrics affected

- Largest Contentful Paint (LCP)
- First Contentful Paint (FCP)
- Speed Index

Most likely cause

Critical CSS files block rendering during the initial page load.

Likely solution

Inline critical CSS, defer non-critical stylesheets, and eliminate render-blocking resources where possible.

Build Analysis

JavaScript

How is the JavaScript bundled?

The website uses **multiple JavaScript bundles (chunk-based code splitting)** rather than a single large bundle. JavaScript is divided into many hashed files, indicating that a modern bundler is splitting the application into smaller chunks.

Is this the right decision?

Yes, Using multiple bundles improves browser caching, reduces the amount of JavaScript downloaded initially, and allows only the required code to be loaded for the current page. This is considered good modern practice.

Is there unused JavaScript?

Yes, The Coverage report shows that a significant portion of the downloaded JavaScript is unused during the initial page load. Several large bundles contain over **80% unused code**, while some chunks approach **99% unused**. Some third-party scripts, including Stripe and Google Tag Manager, also contain substantial amounts of unused code.

Although some of this code may be required later during user interactions, the results indicate there is potential to further optimize the initial JavaScript delivered to users.

Analyze the bundles. Are they good?

Overall, the JavaScript bundling strategy follows modern best practices.

Positive observations include:

- JavaScript is split into multiple smaller chunks rather than one large bundle.
- Bundle sizes are generally moderate.
- HTTP/2 and HTTP/3 are used to efficiently download multiple files in parallel.

However, the analysis also identifies opportunities for improvement:

- Several large bundles contain a high percentage of unused code during the initial page load.
- Numerous third-party scripts (Google Analytics, Google Tag Manager, Google Ads, Facebook Pixel, Quantum Metric, Stripe) increase the amount of JavaScript downloaded and executed before the page becomes fully interactive.

Are source maps available? Should they be?

No public source maps were detected.

This is considered good practice for a production website because it prevents exposure of the original application source code while slightly reducing production asset size.

CSS

How is the CSS bundled?

The website uses **multiple CSS bundles (chunk-based CSS splitting)** rather than a single stylesheet. Numerous hashed CSS files are downloaded, indicating that the CSS has been divided into smaller chunks generated by the application's build process.

This approach allows browsers to cache individual stylesheets more efficiently and avoids downloading one large CSS file.

Is this the right decision?

Yes, Splitting CSS into multiple smaller files is generally considered good practice because it:

- reduces the size of the initial stylesheet,
- improves browser caching,
- allows different pages or components to load only the styles they require,
- avoids maintaining one very large CSS file.

The CSS files shown are also relatively small, with most ranging between **2 kB and 25 kB**, while the largest files remain below **180 kB**.

Is there unused CSS?

Yes, The Coverage report indicates that a very large percentage of downloaded CSS is **unused during the initial page load**.

Examples include:

File	Unused CSS
72069933491cdcd1.css	**97.3%**
867fe5d528c78b88.css	**97.5%**
c560c0aa2a63fe41.css	**99.1%**
de2009abb17e7acd.css	**98.3%**
09b20b591cad409e.css	**97.4%**

This suggests that many styles are downloaded before they are actually needed.

Some unused CSS is expected because different components are loaded later or are only displayed after user interaction. However, consistently high unused percentages (often **95–99%**) indicate there is potential to further optimize CSS delivery.

IMAGES

1. Are the images shipped in multiple sizes/formats?

Yes, The website serves images in **modern WebP format**, as shown by the **Content-Type: image/webp** response header.

The image URLs also contain resizing parameters such as:

resize-h725-w960

which indicates that images are dynamically resized before being delivered. Rather than always serving the original image, the CDN generates an appropriately sized version for the requested dimensions.

This demonstrates that the website ships images in multiple sizes rather than always serving the original file.

---

2. Is this the right decision?

Yes, This is considered a good optimization strategy because:

- Images are served in the **WebP** format, which is generally smaller than JPEG or PNG while maintaining good visual quality.
- Images are resized by the CDN before being downloaded, reducing unnecessary bandwidth usage.
- Only the required image resolution is transferred to the browser, improving loading performance.

Overall, the image delivery strategy follows modern web performance best practices.

---

3. Is the full-resolution file exposed?

No evidence suggests that the original full-resolution image is exposed.

The requested image URL includes dynamic resizing parameters:

resize-h725-w960

and the server responds with:

Content-Type: image/webp

rather than serving the original `.jpg` file directly.

This indicates that users receive an optimized, resized version of the image instead of the original high-resolution asset.

Third Party Resources

1. What third-party tools are used?

Identified third-party services:

Third-Party Tool	Purpose
Google Analytics (GA4 & Universal Analytics)	Website analytics and visitor tracking.
Google Tag Manager (GTM)	Loads and manages marketing and analytics tags.
Google Ads (AW)	Advertising conversion tracking.
Facebook Pixel (fbevents.js)	Advertising and conversion tracking for Facebook.
Quantum Metric	User experience monitoring and session analytics.
Stripe	Payment processing functionality.

---

2. How are they loaded?

Most third-party resources are **loaded during the initial page load** rather than being delayed or lazy-loaded.

Evidence includes:

- Google Tag Manager scripts
- Google Analytics
- Google Ads
- Facebook Pixel
- Quantum Metric
- Stripe

These requests appear immediately after the page begins loading, indicating that they contribute to the initial network activity.

---

3. How much is each one impacting page load or other performance metrics?

The third-party services increase:

- the total number of network requests,
- the amount of JavaScript downloaded,
- JavaScript execution on the main thread,
- Total Blocking Time (TBT),
- overall page loading time.

The Coverage report also shows that several third-party scripts contain a significant amount of unused JavaScript during the initial page load.

Examples include:

- Stripe (v3) – approximately **73% unused**
- Google Tag Manager scripts – approximately **35–75% unused**
- Quantum Metric – approximately **46% unused**

Although these scripts provide useful functionality, they contribute additional JavaScript that is not immediately required during the initial rendering of the homepage.

---

4. Do any seem inappropriate or unnecessary?

No third-party service appears inherently inappropriate for an e-commerce website. Analytics, advertising, payment processing, and user behavior monitoring are all common components of large online retailers.

However, several tracking and analytics scripts are loaded during the initial page load, even though the Coverage analysis indicates that a substantial portion of their JavaScript is unused initially. Deferring or lazy-loading non-essential third-party scripts could reduce JavaScript execution and improve page performance without removing their functionality.

Corrective Findings

Finding 1 – Large Amount of Unused JavaScript

The Coverage report shows that several JavaScript bundles contain between **80% and 99% unused code** during the initial page load. Reducing unused JavaScript through additional code splitting or deferred loading could improve loading performance and reduce Total Blocking Time.

Finding 2 – Large Amount of Unused CSS

The Coverage report indicates that many CSS files contain **95–99% unused styles** during the initial page load. Loading only the CSS required for the current page would reduce unnecessary downloads and improve rendering efficiency.

Finding 3 – Third-Party JavaScript Loaded Up Front

Several third-party services, including Google Analytics, Google Tag Manager, Facebook Pixel, Quantum Metric, and Stripe, are loaded during the initial page load. Deferring non-essential third-party scripts could reduce JavaScript execution time and improve page responsiveness without removing required functionality.